

Spring Data Extensions

Spring Data Extensions

Querydsl Extension

Setting up Annotation Processing

Web support

Basic Web Support

Creating JSON representations for Page

Spring Data Jackson Modules

Web Databinding Support

Querydsl Web Support

Repository Populators

This section documents a set of Spring Data extensions that enable Spring Data usage in a variety of contexts. Currently, most of the integration is targeted towards Spring MVC.

Querydsl Extension

[Querydsl](#) is a framework that enables the construction of statically typed SQL-like queries through its fluent API.

NOTE

Querydsl maintenance has slowed down to a point where the community has forked the project under OpenFeign at github.com/OpenFeign/querydsl (groupId `io.github.openfeign.querydsl`). Spring Data supports the fork on a best-effort basis.

Several Spring Data modules offer integration with Querydsl through `QuerydslPredicateExecutor`, as the following example shows:

QuerydslPredicateExecutor interface

```
public interface QuerydslPredicateExecutor<T> {  
  
    Optional<T> findById(Predicate predicate);    1  
  
    Iterable<T> findAll(Predicate predicate);    2
```

JAVA

```

    long count(Predicate predicate);           3

    boolean exists(Predicate predicate);       4

    // ... more functionality omitted.
}

```

- 1 Finds and returns a single entity matching the Predicate .
- 2 Finds and returns all entities matching the Predicate .
- 3 Returns the number of entities matching the Predicate .
- 4 Returns whether an entity that matches the Predicate exists.

To use the Querydsl support, extend `QuerydslPredicateExecutor` on your repository interface, as the following example shows:

Querydsl integration on repositories

```

interface UserRepository extends CrudRepository<User, Long>,
    QuerydslPredicateExecutor<User> {
}

```

JAVA

The preceding example lets you write type-safe queries by using Querydsl Predicate instances, as the following example shows:

```

Predicate predicate = user.firstname.equalsIgnoreCase("dave")
    .and(user.lastname.startsWithIgnoreCase("mathews"));

userRepository.findAll(predicate);

```

JAVA

Setting up Annotation Processing

To use Querydsl with Spring Data JPA, you need to set up annotation processing in your build system that generates the Q classes. While you could write the Q classes by hand, it is recommended to use the Querydsl annotation processor to generate them for you to keep your Q classes in sync with your domain model.

Most Spring Data users do not use Querydsl, so it does not make sense to require additional mandatory dependencies for projects that would not benefit from Querydsl. Hence, you need to activate annotation processing in your build system.

The following example shows how to set up annotation processing by mentioning dependencies and compiler config changes in Maven and Gradle:

Maven	Gradle	Maven (OpenFeign)	Gradle (OpenFeign)
-------	--------	-------------------	--------------------

XML

```

<dependencies>
  <dependency>
    <groupId>com.querydsl</groupId>
    <artifactId>querydsl-jpa</artifactId>
    <version>${querydslVersion}</version>
    <classifier>jakarta</classifier>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <annotationProcessorPaths>
          <!-- Explicit opt-in required via annotationProcessors or
annotationProcessorPaths on Java 22+, see
https://bugs.openjdk.org/browse/JDK-8306819 -->
          <annotationProcessorPath>
            <groupId>com.querydsl</groupId>
            <artifactId>querydsl-apt</artifactId>
            <version>${querydslVersion}</version>
            <classifier>jakarta</classifier>
          </annotationProcessorPath>
          <annotationProcessorPath>
            <groupId>jakarta.persistence</groupId>
            <artifactId>jakarta.persistence-api</artifactId>
          </annotationProcessorPath>
        </annotationProcessorPaths>

        <!-- Recommended: Some IDE's might require this configuration to include
generated sources for IDE usage -->
        <generatedTestSourcesDirectory>target/generated-test-
sources</generatedTestSourcesDirectory>
        <generatedSourcesDirectory>target/generated-
sources</generatedSourcesDirectory>
      </configuration>
    </plugin>
  </plugins>
</build>

```

Note that the setup above shows the simplest usage omitting any other options or dependencies that your project might require.

Web support

Spring Data modules that support the repository programming model ship with a variety of web support. The web related components require Spring MVC JARs to be on the classpath. Some of them even provide integration with [Spring HATEOAS](#). In general, the integration support is enabled by using the `@EnableSpringDataWebSupport` annotation in your JavaConfig configuration class, as the following example shows:

Enabling Spring Data web support

Java	XML
------	------------

```
@Configuration
@EnableWebMvc
@EnableSpringDataWebSupport
class WebConfiguration {}
```

JAVA

The `@EnableSpringDataWebSupport` annotation registers a few components. We discuss those later in this section. It also detects Spring HATEOAS on the classpath and registers integration components (if present) for it as well.

Basic Web Support

Enabling Spring Data web support in XML

The configuration shown in the previous section registers a few basic components:

- A Using the [DomainClassConverter](#) Class to let Spring MVC resolve instances of repository-managed domain classes from request parameters or path variables.
- [HandlerMethodArgumentResolver](#) implementations to let Spring MVC resolve `Pageable` and `Sort` instances from request parameters.
- Jackson Modules to de-/serialize types like `Point` and `Distance` , or store specific ones, depending on the Spring Data Module used.

Using the `DomainClassConverter` Class

The `DomainClassConverter` class lets you use domain types in your Spring MVC controller method signatures directly so that you need not manually lookup the instances through the repository, as the following example shows:

A Spring MVC controller using domain types in method signatures

```

@Controller
@RequestMapping("/users")
class UserController {

    @RequestMapping("/{id}")
    String showUserForm(@PathVariable("id") User user, Model model) {

        model.addAttribute("user", user);
        return "userForm";
    }
}

```

The method receives a `User` instance directly, and no further lookup is necessary. The instance can be resolved by letting Spring MVC convert the path variable into the `id` type of the domain class first and eventually access the instance through calling `findById(...)` on the repository instance registered for the domain type.

NOTE

Currently, the repository has to implement `CrudRepository` to be eligible to be discovered for conversion.

HandlerMethodArgumentResolvers for Pageable and Sort

The configuration snippet shown in the previous section also registers a `PageableHandlerMethodArgumentResolver` as well as an instance of `SortHandlerMethodArgumentResolver`. The registration enables `Pageable` and `Sort` as valid controller method arguments, as the following example shows:

Using Pageable as a controller method argument

```

@Controller
@RequestMapping("/users")
class UserController {

    private final UserRepository repository;

    UserController(UserRepository repository) {
        this.repository = repository;
    }

    @RequestMapping
    String showUsers(Model model, Pageable pageable) {

        model.addAttribute("users", repository.findAll(pageable));
        return "users";
    }
}

```

```

    }
}

```

The preceding method signature causes Spring MVC try to derive a `Pageable` instance from the request parameters by using the following default configuration:

Table 1. Request parameters evaluated for `Pageable` instances

page	Page you want to retrieve. 0-indexed and defaults to 0.
size	Size of the page you want to retrieve. Defaults to 20.
sort	Properties that should be sorted by in the format <code>property,property(,ASC DESC),IgnoreCase</code> . The default sort direction is case-sensitive ascending. Use multiple <code>sort</code> parameters if you want to switch direction or case sensitivity — for example, <code>?sort=firstname&sort=lastname,asc&sort=city,ignorecase</code> .

To customize this behavior, register a bean that implements the `PageableHandlerMethodArgumentResolverCustomizer` interface or the `SortHandlerMethodArgumentResolverCustomizer` interface, respectively. Its `customize()` method gets called, letting you change settings, as the following example shows:

```

@Bean SortHandlerMethodArgumentResolverCustomizer sortCustomizer() {
    return s -> s.setPropertyDelimiter("<-->");
}

```

JAVA

If setting the properties of an existing `MethodArgumentResolver` is not sufficient for your purpose, extend either `SpringDataWebConfiguration` or the HATEOAS-enabled equivalent, override the `pageableResolver()` or `sortResolver()` methods, and import your customized configuration file instead of using the `@Enable` annotation.

If you need multiple `Pageable` or `Sort` instances to be resolved from the request (for multiple tables, for example), you can use Spring's `@Qualifier` annotation to distinguish one from another. The request parameters then have to be prefixed with `${qualifier}_`. The following example shows the resulting method signature:

```

String showUsers(Model model,
    @Qualifier("thing1") Pageable first,
    @Qualifier("thing2") Pageable second) { ... }

```

JAVA

You have to populate `thing1_page`, `thing2_page`, and so on.

The default `Pageable` passed into the method is equivalent to a `PageRequest.of(0, 20)`, but you can customize it by using the `@PageableDefault` annotation on the `Pageable` parameter.

Creating JSON representations for Page

It's common for Spring MVC controllers to try to ultimately render a representation of a Spring Data page to clients. While one could simply return `Page` instances from handler methods to let Jackson render them as is, we strongly recommend against this as the underlying implementation class `PageImpl` is a domain type. This means we might want or have to change its API for unrelated reasons, and such changes might alter the resulting JSON representation in a breaking way.

With Spring Data 3.1, we started hinting at the problem by issuing a warning log describing the problem. We still ultimately recommend to leverage the integration with Spring HATEOAS for a fully stable and hypermedia-enabled way of rendering pages that easily allow clients to navigate them. But as of version 3.3 Spring Data ships a page rendering mechanism that is convenient to use but does not require the inclusion of Spring HATEOAS.

Using Spring Data' PagedModel

At its core, the support consists of a simplified version of Spring HATEOAS' `PagedModel` (the Spring Data one located in the `org.springframework.data.web` package). It can be used to wrap `Page` instances and result in a simplified representation that reflects the structure established by Spring HATEOAS but omits the navigation links.

JAVA

```
@Controller
class MyController {

    private final MyRepository repository;

    // Constructor omitted

    @GetMapping("/page")
    PagedModel<?> page(Pageable pageable) {
        return new PagedModel<>(repository.findAll(pageable)); 1
    }
}
```

1 Wraps the `Page` instance into a `PagedModel`.

This will result in a JSON structure looking like this:

JAVASCRIPT

```
{
  "content" : [
```

```

    ... // Page content rendered here
  ],
  "page" : {
    "size" : 20,
    "totalElements" : 30,
    "totalPages" : 2,
    "number" : 0
  }
}

```

Note how the document contains a `page` field exposing the essential pagination metadata.

Globally enabling simplified Page rendering

If you don't want to change all your existing controllers to add the mapping step to return `PagedModel` instead of `Page` you can enable the automatic translation of `PageImpl` instances into `PagedModel` by tweaking `@EnableSpringDataWebSupport` as follows:

```

@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)
class MyConfiguration { }

```

JAVA

This will allow your controller to still return `Page` instances and they will automatically be rendered into the simplified representation:

```

@Controller
class MyController {

    private final MyRepository repository;

    // Constructor omitted

    @GetMapping("/page")
    Page<?> page(Pageable pageable) {
        return repository.findAll(pageable);
    }
}

```

JAVA

Hypermedia Support for Page and Slice

Spring HATEOAS ships with a representation model class (`PagedModel` / `SlicedModel`) that allows enriching the content of a `Page` or `Slice` instance with the necessary `Page` / `Slice` metadata as well as links to let the clients easily navigate the pages. The conversion of a `Page` to a `PagedModel` is done by

an implementation of the Spring HATEOAS `RepresentationModelAssembler` interface, called the `PagedResourcesAssembler`. Similarly `Slice` instances can be converted to a `SlicedModel` using a `SlicedResourcesAssembler`. The following example shows how to use a `PagedResourcesAssembler` as a controller method argument, as the `SlicedResourcesAssembler` works exactly the same:

Using a `PagedResourcesAssembler` as controller method argument

```
@Controller
class PersonController {

    private final PersonRepository repository;

    // Constructor omitted

    @GetMapping("/people")
    ResponseEntity<PagedModel<Person>> people(Pageable pageable,
        PagedResourcesAssembler assembler) {

        Page<Person> people = repository.findAll(pageable);
        return ResponseEntity.ok(assembler.toModel(people));
    }
}
```

JAVA

Enabling the configuration, as shown in the preceding example, lets the `PagedResourcesAssembler` be used as a controller method argument. Calling `toModel(...)` on it has the following effects:

- The content of the `Page` becomes the content of the `PagedModel` instance.
- The `PagedModel` object gets a `PageMetadata` instance attached, and it is populated with information from the `Page` and the underlying `Pageable`.
- The `PagedModel` may get `prev` and `next` links attached, depending on the page's state. The links point to the URI to which the method maps. The pagination parameters added to the method match the setup of the `PageableHandlerMethodArgumentResolver` to make sure the links can be resolved later.

Assume we have 30 `Person` instances in the database. You can now trigger a request (`GET localhost:8080/people`) and see output similar to the following:

```
{ "links" : [
  { "rel" : "next", "href" : "http://localhost:8080/persons?page=1&size=20" }
],
  "content" : [
    ... // 20 Person instances rendered here
  ],
}
```

JAVASCRIPT

```
"page" : {  
  "size" : 20,  
  "totalElements" : 30,  
  "totalPages" : 2,  
  "number" : 0  
}  
}
```

| WARNING

The JSON envelope format shown here doesn't follow any formally specified structure and it's not guaranteed stable and we might change it at any time. It's highly recommended to enable the rendering as a hypermedia-enabled, official media type, supported by Spring HATEOAS, like [HAL](#). Those can be activated by using its `@EnableHypermediaSupport` annotation. Find more information in the [Spring HATEOAS reference documentation](#).

The assembler produced the correct URI and also picked up the default configuration to resolve the parameters into a `Pageable` for an upcoming request. This means that, if you change that configuration, the links automatically adhere to the change. By default, the assembler points to the controller method it was invoked in, but you can customize that by passing a custom `Link` to be used as base to build the pagination links, which overloads the `PagedResourcesAssembler.toModel(...)` method.

Spring Data Jackson Modules

The core module, and some of the store specific ones, ship with a set of Jackson Modules for types, like `org.springframework.data.geo.Distance` and `org.springframework.data.geo.Point`, used by the Spring Data domain.

Those modules are imported once web support is enabled and `tools.jackson.databind.ObjectMapper` is available.

During initialization `SpringDataJackson3Modules`, like the `SpringDataJackson3Configuration`, get picked up by the infrastructure, so that the declared `tools.jackson.databind.JsonModule`s are made available to the Jackson `ObjectMapper`.

Data binding mixins for the following domain types are registered by the common infrastructure.

```
org.springframework.data.geo.Distance  
org.springframework.data.geo.Point  
org.springframework.data.geo.Box  
org.springframework.data.geo.Circle  
org.springframework.data.geo.Polygon
```

| NOTE

NOTE

The individual module may provide additional `SpringDataJackson3Modules`. Please refer to the store specific section for more details.

NOTE

Jackson 2 support is deprecated and will be removed in a future release.

Web Databinding Support

You can use Spring Data projections (described in [Projections](#)) to bind incoming request payloads by using either [JSONPath](#) expressions (requires [Jayway JsonPath](#)) or [XPath](#) expressions (requires [XmlBeam](#)), as the following example shows:

HTTP payload binding using JSONPath or XPath expressions

JAVA

```
@ProjectedPayload
public interface UserPayload {

    @XBRead("//firstname")
    @JsonPath("$.firstname")
    String getFirstname();

    @XBRead("/lastname")
    @JsonPath({ "$.lastname", "$.user.lastname" })
    String getLastName();
}
```

You can use the type shown in the preceding example as a Spring MVC handler method argument or by using `ParameterizedTypeReference` on one of methods of the `RestTemplate`. The preceding method declarations would try to find `firstname` anywhere in the given document. The `lastname` XML lookup is performed on the top-level of the incoming document. The JSON variant of that tries a top-level `lastname` first but also tries `lastname` nested in a `user` sub-document if the former does not return a value. That way, changes in the structure of the source document can be mitigated easily without having clients calling the exposed methods (usually a drawback of class-based payload binding).

Nested projections are supported as described in [Projections](#). If the method returns a complex, non-interface type, a Jackson `ObjectMapper` is used to map the final value.

For Spring MVC, the necessary converters are registered automatically as soon as `@EnableSpringDataWebSupport` is active and the required dependencies are available on the classpath. For usage with `RestTemplate`, register a `ProjectingJacksonHttpMessageConverter` (JSON) or `XmlBeamHttpMessageConverter` manually.

For more information, see the [web projection example](#) in the canonical [Spring Data Examples repository](#).

Querydsl Web Support

For those stores that have [Querydsl](#) integration, you can derive queries from the attributes contained in a `Request` query string.

Consider the following query string:

```
?firstname=Dave&lastname=Matthews
```

TEXT

Given the `User` object from the previous examples, you can resolve a query string to the following value by using the `QuerydslPredicateArgumentResolver`, as follows:

```
QUser.user.firstname.eq("Dave").and(QUser.user.lastname.eq("Matthews"))
```

TEXT

NOTE

The feature is automatically enabled, along with `@EnableSpringDataWebSupport`, when `Querydsl` is found on the classpath.

Adding a `@QuerydslPredicate` to the method signature provides a ready-to-use `Predicate`, which you can run by using the `QuerydslPredicateExecutor`.

TIP

Type information is typically resolved from the method's return type. Since that information does not necessarily match the domain type, it might be a good idea to use the `root` attribute of `QuerydslPredicate`.

The following example shows how to use `@QuerydslPredicate` in a method signature:

```
@Controller
class UserController {

    @Autowired UserRepository repository;

    @RequestMapping(value = "/", method = RequestMethod.GET)
    String index(Model model, @QuerydslPredicate(root = User.class) Predicate predicate,
1         Pageable pageable, @RequestParam MultiValueMap<String, String> parameters) {

        model.addAttribute("users", repository.findAll(predicate, pageable));

        return "index";
    }
}
```

JAVA

```
}  
}
```

- 1 Resolve query string arguments to matching Predicate for User.

The default binding is as follows:

- Object on simple properties as eq.
- Object on collection like properties as contains.
- Collection on simple properties as in.

You can customize those bindings through the bindings attribute of @QuerydslPredicate or by making use of Java 8 default methods and adding the QuerydslBinderCustomizer method to the repository interface, as follows:

```
interface UserRepository extends CrudRepository<User, String>, JAVA  
    QuerydslPredicateExecutor<User>, 1  
    QuerydslBinderCustomizer<QUser> { 2  
  
    @Override  
    default void customize(QuerydslBindings bindings, QUser user) {  
  
        bindings.bind(user.username).first((path, value) -> path.contains(value)) 3  
        bindings.bind(String.class)  
            .first((StringPath path, String value) -> path.containsIgnoreCase(value)); 4  
        bindings.excluding(user.password); 5  
    }  
}
```

- 1 QuerydslPredicateExecutor provides access to specific finder methods for Predicate.
- 2 QuerydslBinderCustomizer defined on the repository interface is automatically picked up and shortcuts @QuerydslPredicate(bindings=...).
- 3 Define the binding for the username property to be a simple contains binding.
- 4 Define the default binding for String properties to be a case-insensitive contains match.
- 5 Exclude the password property from Predicate resolution.

TIP

You can register a QuerydslBinderCustomizerDefaults bean holding default Querydsl bindings before applying specific bindings from the repository or @QuerydslPredicate.

Repository Populators

If you work with the Spring JDBC module, you are probably familiar with the support for populating a `DataSource` with SQL scripts. A similar abstraction is available on the repositories level, although it does not use SQL as the data definition language because it must be store-independent. Thus, the populators support XML (through Spring's OXM abstraction) and JSON (through Jackson) to define data with which to populate the repositories.

Assume you have a file called `data.json` with the following content:

Data defined in JSON

JAVASCRIPT

```
[ { "_class" : "com.acme.Person",  
  "firstname" : "Dave",  
  "lastname" : "Matthews" },  
  { "_class" : "com.acme.Person",  
    "firstname" : "Carter",  
    "lastname" : "Beauford" } ]
```

You can populate your repositories by using the populator elements of the repository namespace provided in Spring Data Commons. To populate the preceding data to your `PersonRepository`, declare a populator similar to the following:

Declaring a Jackson repository populator

XML

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xmlns:repository="http://www.springframework.org/schema/data/repository"  
  xsi:schemaLocation="http://www.springframework.org/schema/beans  
    https://www.springframework.org/schema/beans/spring-beans.xsd  
    http://www.springframework.org/schema/data/repository  
    https://www.springframework.org/schema/data/repository/spring-repository.xsd">  
  
  <repository:jackson2-populator locations="classpath:data.json" />  
  
</beans>
```

The preceding declaration causes the `data.json` file to be read and deserialized by a Jackson `ObjectMapper`.

The type to which the JSON object is unmarshalled is determined by inspecting the `_class` attribute of the JSON document. The infrastructure eventually selects the appropriate repository to handle the object that was deserialized.

To instead use XML to define the data the repositories should be populated with, you can use the `unmarshaller-populator` element. You configure it to use one of the XML marshaller options available

in Spring OXM. See the [Spring reference documentation](#) for details. The following example shows how to unmarshall a repository populator with JAXB:

Declaring an unmarshalling repository populator (using JAXB)

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:repository="http://www.springframework.org/schema/data/repository"
  xmlns:oxm="http://www.springframework.org/schema/oxm"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/data/repository
    https://www.springframework.org/schema/data/repository/spring-repository.xsd
    http://www.springframework.org/schema/oxm
    https://www.springframework.org/schema/oxm/spring-oxm.xsd">

  <repository:unmarshaller-populator locations="classpath:data.json"
    unmarshaller-ref="unmarshaller" />

  <oxm:jaxb2-marshaller contextPath="com.acme" />

</beans>
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.